

STAR Statistic Vocabularies Parsing and Structuring

Luca Foresti

2026-06-25

Repository Details & Overview

This report follows the numbering of the STAR statistical-vocabulary assignment and documents the current implementation on the 2,000-table Eurostat STAR subset. All quantitative results in the main report are from `configs/core.yaml`, whose configured corpus is `eurostat_2000_tables.tgz` with expected table count 2,000. The larger 7,605-table corpus is treated as scale work and is not used for the reported figures.

Repository link: <https://github.com/Forest904/statistical-vocabulary-discovery>

Installation is documented in `README.md`. The core setup is:

```
git clone https://github.com/Forest904/statistical-vocabulary-discovery.git
cd statistical-vocabulary-discovery
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -e ".[dev,notebooks,ml,api]"
```

External resources are recorded in `data/processed/resource_manifest.json` and summarized in `report/resource_register.md`. They include the STAR 2,000-table archive from Zenodo record 15681384, the table-title catalog, Eurostat NUTS 2024 geography data, the Eurostat GEO codelist used by the enhanced geography variant, STAR retrieval resources, and the PEARL-small embedding model. Prompt templates are stored separately in `prompts/classify_term.md`, `prompts/label_cluster.md`, and `prompts/classify_relation.md`. Paid LLM calls are optional and disabled for the submitted core run.

Current core run IDs and snapshots:

Stage	Run ID
Submitted validated category export	run_42d33d7967059d3e8f22
Latest local-hybrid classifier rerun	run_d96fee3c3c6166fddbac

Stage	Run ID
Current clustering export	run_af85e9400741e5dc8d39
Current relationships export	run_1b4d6642319427ea9aa0
Current search index	run_4e3c9bffe2048ae27b2

`report/core_release_manifest.json` is the frozen checksum snapshot of the current professor-facing artifacts.

Required assignment outputs:

Output	Count
outputs/measures.csv	2,894
outputs/dimension_names.csv	359
outputs/dimension_values.csv	607
outputs/units.csv	261

Supplementary outputs:

Output	Count	Role
outputs/other_ambiguous.csv	9,114	Justified extra category for unsafe terms
outputs/measure_clusters.csv	2,894	Step 7 domain grouping
outputs/measure_relations.csv	30,695	Step 8 bonus candidate graph

The complete core workflow produces reproducible extraction artifacts, required assignment CSVs, validation metrics, and supplementary search/graph deliverables from the same 2,000-table corpus. Figure 1 summarizes that flow; every figure in this report is generated from existing core artifacts by `scripts/generate_report_assets.py`.

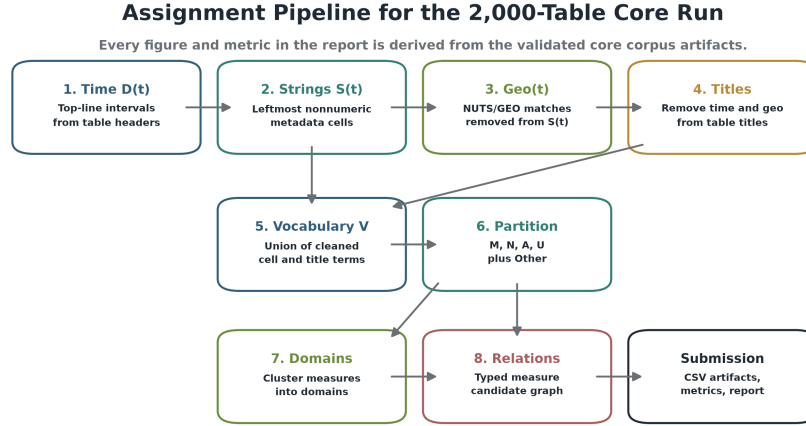


Figure 1. Assignment pipeline for the validated 2,000-table core submission.

1. Extraction of Time Intervals, $D(t)$

Methodology. The extractor reads the parsed table inventory, uses the detected metadata/time-column boundary, and scans time-header cells on the top line of each table. It recognizes years, integer-like year values such as 2024.0, quarters, month labels, ISO-style dates, explicit year ranges, and selected title time phrases needed for title cleaning. Rough pattern matching is intentionally preferred over inference: for example, year-like values are accepted only when they satisfy configured bounds and context checks.

Data granularity. Each accepted expression is normalized to a typed interval with raw text, start date, end date, granularity, source area, source location or title span, rule ID, confidence, and deterministic `time_id`. Supported granularities include years, quarters, months, days, and ranges.

Output and evaluation. The primary artifact is `data/processed/table_times.parquet`. On the 50-table extraction review sample, `report/extraction_metrics.json` reports time precision 1.000, recall 1.000, and F1 1.000.

Limitations. Ambiguous or unsupported period expressions are left unparsed rather than guessed. This makes errors visible in review instead of silently adding questionable time intervals.

Code reference. Temporal extraction is implemented in `src/statvocab/time_extract.py`; title-time removal used in step 4 is implemented in `src/statvocab/title_extract.py`. Regression coverage is in `tests/test_time_extract.py`.

2. Extraction of Leftmost Column Strings, $S(t)$

Methodology. The ingestion stage detects contiguous metadata columns before the time columns. String extraction reopens each parsed source table from `parsed_local_path`, collects metadata-column names as header evidence, and collects non-blank, non-missing, non-numeric metadata values as candidate strings. Observation cells and one-letter observation flags are excluded.

Constraints implementation. $S(t)$ is duplicate-free because terms are normalized to matching keys before table-level aggregation. It is disjoint from $D(t)$ because recognized time-header values are routed to the time artifact and are not emitted as leftmost-column vocabulary strings.

Output and evaluation. The primary artifact is `data/processed/table_strings.parquet`. Ingestion inventoried exactly 2,000 tables: 1,771 parsed cleanly, 229 parsed with warnings, and zero failed. On the reviewed extraction sample, string precision, recall, and F1 are all 1.000.

Limitations. Malformed rows are skipped consistently with ingestion diagnostics rather than repaired silently. This favors reproducibility over speculative table repair.

Code reference. Table parsing and metadata-boundary handling are in `src/statvocab/ingest.py`; vocabulary occurrence construction is in `src/statvocab/vocabulary.py`. Relevant tests include `tests/test_ingest.py` and `tests/test_observations.py`.

3. Identification of Geographical Units, $Geo(t)$

Dictionary integration. The assignment-compliant geography baseline uses Eurostat NUTS 2024 code and name matching. The submitted core configuration also supports an enhanced variant that adds the official Eurostat GEO codelist. Accepted matches are exact raw matches, exact normalized matches, or official alias matches; fuzzy geography matches are not accepted.

Vocabulary construction. For each table, accepted geography terms are removed from leftmost-column strings before vocabulary construction, implementing $V(t) = S(t) \setminus \text{setminus } Geo(t)$. The configured core run uses the enhanced geography variant for submitted vocabulary files while preserving the NUTS-only baseline as an auditable method variant.

Output and evaluation. The primary artifact is `data/processed/table_geographies.parquet`. On the reviewed extraction sample, geography precision, recall, and F1 are all 1.000.

Limitations. Non-NUTS aggregates and geography expressions absent from the official dictionaries may remain unresolved. The implementation does not invent geography labels to cover those cases.

Code reference. NUTS and Eurostat GEO matching are implemented in `src/statvocab/geo_extract.py`; resource acquisition and provenance are handled by `src/statvocab/resources.py`. Regression coverage is in `tests/test_geo_extract.py`.

4. Processing Table Titles, $\tau(t)$

Title retrieval. Table titles are acquired from the provided natural-language title catalog and joined to parsed table IDs during ingestion and vocabulary construction.

Entity separation. Title processing removes recognized time spans and active-variant geography spans using the same conservative temporal and geographic evidence used for table cells. The cleaned full title is preserved as strong measure evidence. Clause splitting is limited to high-confidence separators and parenthetical expressions.

Vocabulary expansion. Cleaned title terms are appended to each table vocabulary $V(t)$ after time and geography removal. The process preserves source text and does not paraphrase, rename, split aggressively, or infer concepts not present in the title.

Output and evaluation. The primary artifact is `data/processed/title_terms.parquet`. On the reviewed extraction sample, title precision, recall, and F1 are all 1.000.

Limitations. Because the method avoids paraphrase, some semantically useful title variants are not generated. This is a deliberate grounding choice.

Code reference. Title cleaning and title-term construction are implemented in `src/statvocab/title_extract.py` and integrated into table vocabulary construction in `src/statvocab/vocabulary.py`. Tests are in `tests/test_vocabulary.py`.

5. Construction of Total Vocabulary, V

Aggregation. The total vocabulary is built as the union of all table vocabularies after removing time and geography evidence: $V = \text{union}_t V(t)$. The implementation combines non-temporal, non-geographic metadata strings with cleaned title terms.

Optimization. Terms are deduplicated with display-preserving normalization: matching keys casefold text, normalize whitespace, normalize compatible dash and apostrophe forms, and strip harmless terminal punctuation while preserving the original display text. Stable term IDs and occurrence IDs are generated from source evidence. This prevents repeated classification work for terms that recur across many tables.

Outputs. The primary artifacts are `data/processed/vocabulary.parquet` for distinct global terms, `data/processed/term_occurrences.parquet` for provenance, and `data/processed/table_vocabulary.parquet` for per-table vocabulary membership.

Limitations. Conflicting source roles are preserved in evidence summaries rather than overwritten. Later stages may route those terms to `other_ambiguous` instead of forcing a category.

Code reference. Normalization is implemented in `src/statvocab/normalize.py`; vocabulary aggregation is implemented in `src/statvocab/vocabulary.py`. Regression coverage is in `tests/test_vocabulary.py`.

6. Vocabulary Categorization and Partitioning

Partitioning methodology. The required partition divides V into measures M, dimension names N, dimension values A, and unit information U. The submitted validated category export is `run_42d33d7967059d3e8f22`. It combines protected high-precision rules, structural evidence features, PEARL-small embeddings, and local model evidence, while routing low-confidence or conflicting outputs to `other_ambiguous`.

Extended category. `other_ambiguous` is used because the assignment allows an extra category when needed. It contains fragmentary terms, conflicting evidence cases, and terms whose category would otherwise require unsafe inference. This category is not one of the four required output files, but it prevents low-confidence terms from polluting M, N, A, or U.

LLM guardrails. Optional paid LLM adjudication is scaffolded through `prompts/classify_term.md`, but it is disabled by default and is not a dependency of the submitted core artifacts. When enabled, the architecture requires strict JSON schema validation, existing `term_id` membership, evidence-ID membership, and category values from the controlled label set. The submitted run has accepted hallucination count 0.

Artifact mapping:

Category	File	Count
Measures M	<code>outputs/measures.csv</code>	2,894
Dimension names N	<code>outputs/dimension_names.csv</code>	359
Dimension values A	<code>outputs/dimension_values.csv</code>	607
Units U	<code>outputs/units.csv</code>	261
Other/ambiguous	<code>outputs/other_ambiguous.csv</code>	9,114

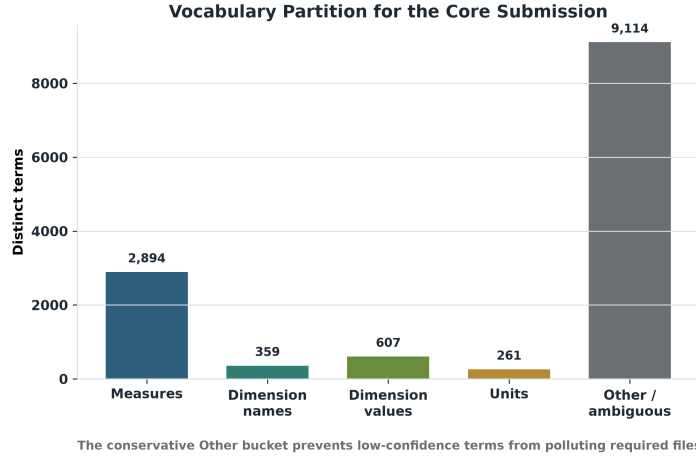


Figure 2. Submitted vocabulary partition, including the justified extra `other_ambiguous` bucket.

Quality evaluation. Evaluation uses `data/gold/vocabulary_gold_labels.csv` with 500 completed random audit labels, `data/gold/vocabulary_gold_relabel.csv` with 50 duplicate labels, `data/gold/vocabulary_reclaim_labels.csv` with 250 targeted reclaim labels, `data/gold/vocabulary_reclaim_relabel.csv` with 25 duplicate targeted rows, and the small compiled human-loop label set when present. Both duplicate audits report raw agreement 1.000 and Cohen’s kappa 1.000. The headline submission metrics use the random audit slice from `report/classification_metrics.json`:

Split	Accuracy	Macro-F1	Weighted-F1
All random rows	0.916	0.869	0.909
Random validation	0.930	0.881	0.916
Random final test	0.930	0.884	0.926

The completed targeted reclaim audit is reported separately as a diagnostic stress test over terms previously assigned to `other_ambiguous`:

Split	Accuracy	Macro-F1	Weighted-F1
All targeted rows	0.204	0.121	0.196
Targeted validation	0.020	0.008	0.001
Targeted final test	0.100	0.051	0.164

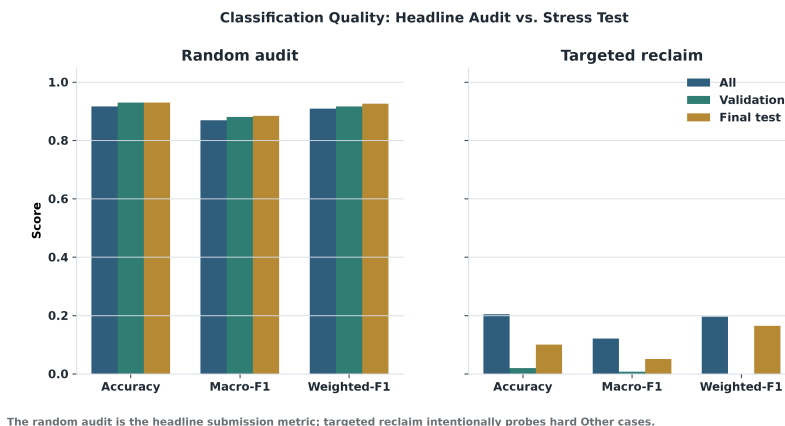


Figure 3. Classification quality on the headline random audit versus the targeted reclaim stress test.

These diagnostic metrics are low because the accepted classifier still abstains heavily on reclaim candidates. That behavior is intentional: the current artifacts prefer high-precision assignment files over forcing ambiguous terms into M, N, A, or U.

Targeted reclaim and human loop. To address the large `other_ambiguous` bucket without overclaiming, the repository includes a completed targeted review: `data/gold/vocabulary_reclaim_sample.csv` and `data/gold/vocabulary_reclaim_labels.csv` contain 250 candidate terms, with 25 blind duplicate rows in `data/gold/vocabulary_reclaim_relabel.csv`. The human-loop review UI adds a continuous route for expanding ground truth and improving later classifier versions. The latest local-hybrid rerun, `run_d96fee3c3c6166fddbdc`, validated against the current vocabulary but was not promoted over the conservative submitted CSVs: targeted final-test non-other precision was $0.714 < 0.850$, targeted reclaim macro-F1 did not improve, `other_ambiguous` reduction was $0.000 < 0.200$, and targeted measure recall remained below the promotion gate.

Limitations. The completed labels were produced through repository audits rather than a fully independent second-human annotation study. Because the latest reclaim attempt failed the acceptance gate, the accepted outputs remain conservative and no automatic reduction of `other_ambiguous` is claimed here.

Code reference. Partition orchestration and acceptance gating are in `src/statvocab/classification.py`; deterministic rules are in `src/statvocab/classify_rules.py`; embedding support is in `src/statvocab/classify_embeddings.py`; targeted review sampling is in `src/statvocab/targeted_review.py`; optional LLM adjudication is in `src/statvocab/classify_llm.py`. Tests include `tests/test_classification_contracts.py`, `tests/test_classify_rules.py`, and `tests/test_grounding.py`.

7. Structuring and Clustering Measures

Clustering approach. Step 7 starts from the final measures in `outputs/measures.csv`. Measures are embedded with the same pinned PEARL-small configuration used in classification, then clustered with HDBSCAN. HDBSCAN noise points are preserved as `unclustered` so unstable assignments remain visible rather than being forced into a domain.

Knowledge sourcing. Domain labels come from a controlled top-level domain vocabulary. For each non-noise cluster, representative measures nearest the cluster centroid are compared with embedded domain descriptions. A domain is assigned only when the best score clears the configured threshold and margin; otherwise the cluster is labeled `cross-domain` or `other`. Agglomerative clustering with cosine distance and average linkage is exported as a baseline.

Outputs. The main clustering artifact is `outputs/measure_clusters.csv`, which contains one row for each of the 2,894 current measures. Supporting artifacts for the current export are in `outputs/clustering/run_af85e9400741e5dc8d39/`: `agglomerative_baseline.csv`, `domain_taxonomy.json`, `manual_cluster_review_sample.csv`, and `clustering_summary.json`.

Quality evaluation. Current metrics from `report/clustering_metrics.json` are:

Metric	Value
Measures	2,894
HDBSCAN clusters	216
Agglomerative baseline clusters	48
Non-noise coverage	0.870
Unclustered measures	376
HDBSCAN stability proxy	0.828
Validation passed	true
Manual review rows completed	217
Mean manual coherence	1.226 / 2
Coherent or strongly coherent	0.668
Domain-label accuracy	0.558
Representative good fraction	0.839

Domain distribution is conservative: `cross-domain` or `other` contains 1,333 measures. Large visible domains include economy and finance, population and demography, labour market, agriculture, health, education, transport, and industry/trade/services. The manual review chart makes the main tuning direction visible: representatives are usually good, but many broad domain labels can be sharpened.

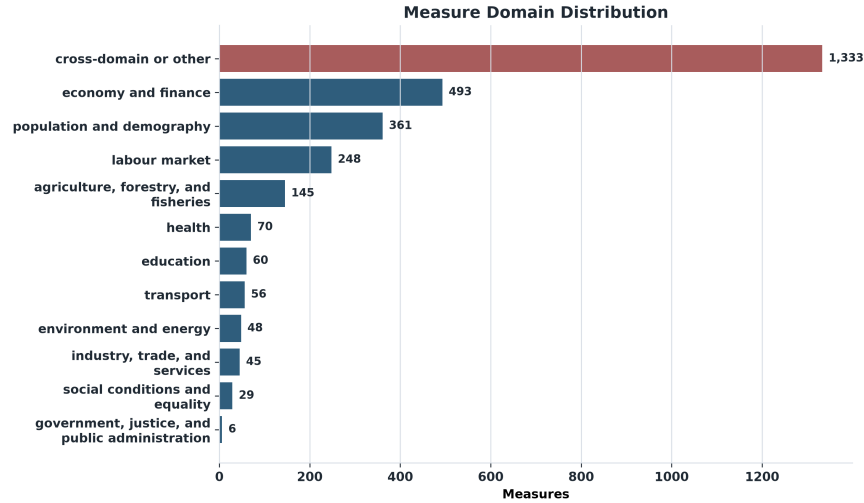


Figure 4. Measure-domain distribution for `outputs/measure_clusters.csv`, with conservative cross-domain assignments highlighted.

Manual review status. The current manual cluster-review file is in the `run_af85e9400741e5dc8d39` clustering directory, and all 217 sampled rows are complete. The review records 121 correct, 24 partial, and 72 wrong domain-label judgments. It also records 182 good and 35 poor representative-quality judgments. This adds human coherence evidence to the structural validation rather than relying only on cluster coverage.

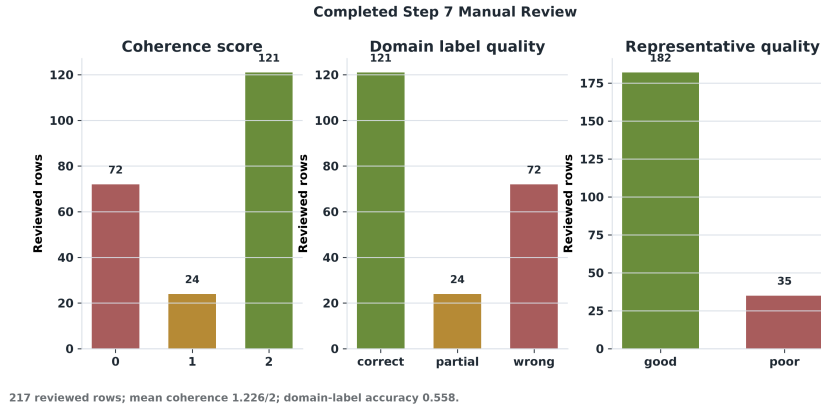


Figure 5. Completed Step 7 manual review summary for coherence, domain-label quality, and representative quality.

Limitations. The current clustering covers most measures but still leaves 376 measures unclustered and routes many assignments to `cross-domain or other`. The manual review shows that many of those conservative cross-domain labels

should be tuned toward more specific domains, especially economy and finance, population and demography, labour market, and industry/trade/services.

Code reference. Measure clustering, baseline export, domain labeling, and manual-review sampling are implemented in `src/statvocab/cluster_measures.py`. Manual-review metric parsing is implemented in `src/statvocab/clustering_evaluate.py`; tests are in `tests/test_cluster_measures.py`.

8. Semantic Relationships Between Measures (Bonus)

Relationship identification. Step 8 is implemented as a bonus candidate graph over final measures. Candidates are generated from embedding similarity, lexical containment, qualifier patterns, title-family signals, and shared domain evidence. Variant and hierarchy evidence take precedence over generic relatedness.

Exported relationship types:

Type	Count
<code>broader_than</code>	16,382
<code>related_to</code>	13,482
<code>variant_of</code>	831

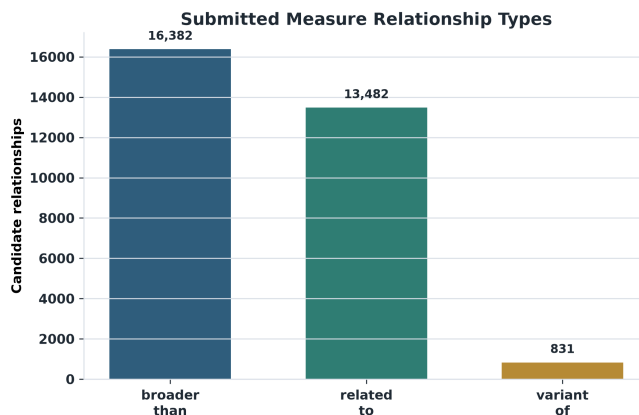


Figure 6. Submitted Step 8 relation type distribution.

`narrower_than` is represented by reading a `broader_than` edge in the inverse direction, so it is not duplicated as a separate row in `outputs/measure_relations.csv`.

Quality evaluation. `report/relations_metrics.json` reports 30,695 submitted candidate relationships and successful structural validation. The current generation run is `run_1b4d6642319427ea9aa0`; it generated 45,351 candidates and exported the 30,695 accepted relations in

`outputs/measure_relations.csv`. Every submitted relation references known final measures, avoids self-relations, avoids duplicate unordered pairs, has an allowed relation type, and includes evidence and confidence fields. Submitted confidence bands include 26,832 candidates from 0.70 to 0.84 and 3,749 from 0.85 to 1.00.

LLM guardrails. Optional LLM relationship adjudication is scaffolded through `prompts/classify_relation.md` and disabled by default. The prompt requires existing endpoint IDs, controlled relation types, evidence-ID membership, self-rejection, and strict JSON.

Manual review status. The 100-row manual relation-review file is `outputs/relations/run_e6e523656ea9857e` and is complete. It reports precision@10 1.000, precision@25 1.000, precision@50 0.960, precision@100 0.890, typed accuracy 0.888, and a false-positive taxonomy dominated by generic qualifier and denominator-fragment matches. The completed sample is used as relation-quality evidence for the broader current relation method; the structural validator covers the refreshed 30,695-row export.

Limitations. The accepted relation graph is structurally valid and manually sampled, but some broad `related_to` edges and lexical-containment false positives remain. Generic qualifier handling and hierarchy rules are the clearest next tightening points.

Code reference. Candidate generation, confidence-filtered export, and full-candidate preservation are implemented in `src/statvocab/relations.py`. Evaluation is implemented in `src/statvocab/relations_evaluate.py`; tests are in `tests/test_relations.py`.

Appendices

A. Resources and Licenses

Resources are recorded in `data/processed/resource_manifest.json` and summarized in `report/resource_register.md`. The core archive is `eurostat_2000_tables.tgz` from Zenodo record 15681384, verified with MD5 97bda36fec3c7c7ed042ee64ab16bf2c. Other resources include the Eurostat title catalog, NUTS 2024 attributes, the Eurostat GEO codelist, STAR questions and annotations, and PEARL-small.

Raw data, external downloads, caches, and generated heavy artifacts are intentionally excluded from git where appropriate. Acquisition commands reproduce them with recorded provenance.

B. Prompts and Model IDs

Prompt files are stored separately:

Prompt	Purpose
<code>prompts/classify_term.md</code>	Optional vocabulary-term adjudication
<code>prompts/label_cluster.md</code>	Optional constrained cluster-domain labeling
<code>prompts/classify_relation.md</code>	Optional relationship adjudication

The local embedding model is Lihuchen/pearl_small; the exact pinned revision is recorded in `configs/core.yaml`. No paid API calls were used for the submitted core artifacts, so monetary LLM cost is 0.

C. Annotation Guide and Agreement

The annotation guide is `report/annotation_guide.md`. Completed vocabulary audit labels are in `data/gold/vocabulary_gold_labels.csv`; duplicate relabel rows are in `data/gold/vocabulary_gold_relabel.csv`.

Agreement on the 50 random duplicate relabel rows is raw agreement 1.000 and Cohen’s kappa 1.000. Agreement on the 25 targeted reclaim duplicate relabel rows is also raw agreement 1.000 and Cohen’s kappa 1.000. Because the labels were completed through repository audits, this is reported as an internal consistency check rather than an independent multi-annotator study.

The targeted reclaim review files are complete: `data/gold/vocabulary_reclaim_labels.csv` contains 250 rows and `data/gold/vocabulary_reclaim_relabel.csv` contains 25 duplicate relabel rows.

D. Reproducibility Commands

Run the core stages:

```

statvocab acquire --config configs/core.yaml
statvocab ingest --config configs/core.yaml
statvocab extract --config configs/core.yaml
statvocab prepare-targeted-review --config configs/core.yaml
statvocab classify --config configs/core.yaml --variant local-hybrid
statvocab cluster-measures --config configs/core.yaml
statvocab relations --config configs/core.yaml
statvocab build-search-index --config configs/core.yaml
statvocab build-knowledge-graph --config configs/core.yaml
statvocab evaluate --config configs/core.yaml --area extraction
statvocab evaluate --config configs/core.yaml --area classification
statvocab evaluate --config configs/core.yaml --area clustering
statvocab evaluate --config configs/core.yaml --area relations
statvocab evaluate --config configs/core.yaml --area retrieval
statvocab benchmark-core --config configs/core.yaml

```

The refreshed core run is also summarized with stage-level wall-clock runtime from `report/performance_metrics.json`.

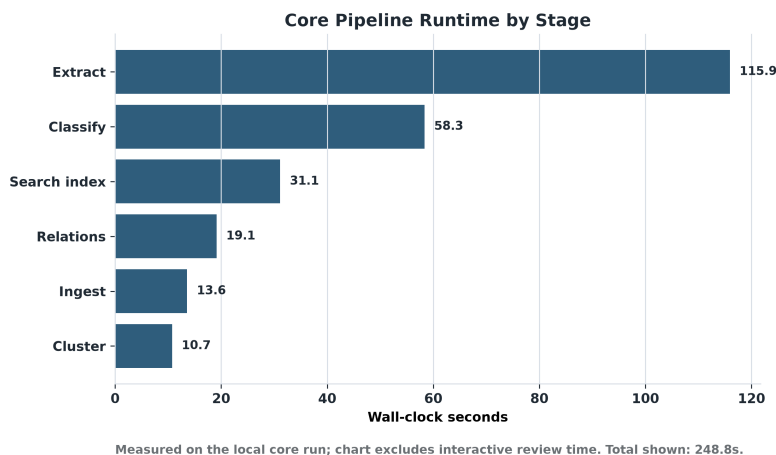


Figure 7. Core pipeline wall-clock runtime by stage.

Validate the submitted artifacts:

```
pytest tests/test_notebooks.py
statvocab validate-artifacts --config configs/core.yaml --run-id run_42d33d7967059d3e8f22
statvocab validate-core-release --config configs/core.yaml
```

E. Supplementary Product: Search, API, Graph, and Review

The search, API, React UI, graph explorer, and review page are supplementary product work and are intentionally presented after the assignment deliverables. The search engine builds one document per source table, returns ranked Eurostat tables with evidence, and does not answer numeric questions. The FastAPI backend exposes grounded search, artifact, relation, graph, and review endpoints; the React frontend provides table search, vocabulary inspection, domain and relation views, a bounded Cytoscape.js graph explorer, and a human-loop review surface.

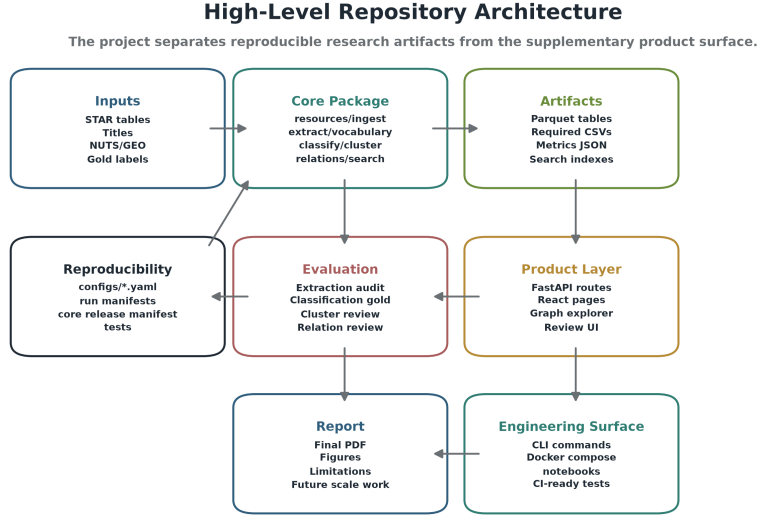


Figure 8. High-level repository architecture separating the core research pipeline from supplementary product surfaces.

Retrieval metrics are in `report/retrieval_metrics.json`; the current search-index run is `run_4e3c9bffffe2048ae27b2`. Review events are continuous improvement data: they expand ground truth for future classifier and relation iterations, but they are not treated as a one-time gate for the submitted assignment artifacts.

Local demo command:

```
docker compose up --build
```

F. Full-Scale Run Status

The full-scale 7,605-table corpus is future scalability work for cloud or larger local hardware. No full-corpus semantic result set is claimed in this report. The 2,000-table subset is the completed, validated submission target because it is the smaller benchmark corpus explicitly provided in the assignment, is reproducible on local hardware, and supports complete extraction, classification, clustering, relationship generation, and evaluation. Full-scale figures and bottleneck analysis should be added only after the large run finishes and its artifacts are validated.